

Ship: Agent Communication Layer

A design note for structured coordination between coding agents, tools, repositories, and humans.

Purpose: define a future communication layer for a larger developer ecosystem where agents can coordinate around tasks, artifacts, decisions, blockers, ownership, and handoffs without devolving into noisy free-form chat.

Core thesis: agents should not primarily chat. They should publish structured facts, artifacts, claims, decisions, blockers, and handoffs. Other agents should query, subscribe to, and act on those durable records.

Primitive	What it means
Task	One named unit of developer workflow, such as <code>create_task_doc</code> , <code>ship_task_doc</code> , <code>fix_ci</code> , <code>review_pr</code> .
Recipe	An ordered composition of task units, such as <code>ship_feature</code> .
Artifact	Durable output from a task: task doc, test results, diff summary, review notes, risk report.
Event	Timeline record: <code>task.created</code> , <code>cursor_run.completed</code> , <code>pr.opened</code> , <code>ci.failed</code> , <code>review.completed</code> .
Claim	Evidence-backed assertion with confidence and references.
Handoff	Structured transfer from one agent/task to another with status, risks, artifacts, and next action.

1. The Product Direction

Ship starts as a personal developer workflow MCP server: task docs, Tower worktrees, Cursor agents, PRs, CI repair, review cycles, and merge readiness. The long-term direction is bigger: make the engineering process executable and observable across agents, tools, repositories, and humans.

The communication layer is the future substrate that lets this ecosystem scale without turning into a chaotic message bus.

What agent awareness should mean

Avoid fuzzy claims like “agents know each other.” The practical version is: agents can discover each other’s outputs, capabilities, decisions, confidence, blockers, and current ownership of work.

- **What work exists?** Current tasks, recipes, runs, PRs, and owners.
- **What has already been tried?** Prior attempts, failed fixes, review cycles, CI repair history.
- **What artifacts exist?** Task docs, review notes, test output, diff summaries, risk assessments.
- **What is blocked?** Missing credentials, unclear requirements, repeated CI failure, human decision needed.
- **Who or what should handle the next step?** Cursor implementer, reviewer agent, CI fixer, release task, or human.

2. Avoid the Naive Message Bus

The tempting design is a general agent chat bus: agent A sends message to agent B, agent B replies, agent C broadcasts updates. That looks powerful, but tends to create noise, duplicated work, and coordination theater.

Instead, the communication layer should be structured and durable. Agents should publish records that future agents can query and trust.

```
Bad primitive:
  send_message(from, to, body)
```

```
Better primitives:
  publish_artifact(...)
  publish_claim(...)
  publish_decision(...)
  publish_handoff(...)
  observe(scope, id)
  route(need, context)
```

3. The Five-Part Communication Layer

3.1 Capability Registry

Agents need to know who is good at what. The system should record capabilities and strengths, not personalities.

```
agents:
  cursor_implementer:
    capabilities: [edit_repo, run_tests, open_pr, fix_ci]
    strengths: [implementation, refactors, small bugfixes]

  architecture_reviewer:
    capabilities: [review_design_doc, review_pr, compare_approaches]
    strengths: [system_design, scope_control, risk_detection]

  test_reviewer:
    capabilities: [inspect_tests, propose_regression_tests, evaluate_coverage]
```

Recipes can then target roles instead of hardcoding one model or one agent. The route layer can choose the right backend: Cursor SDK, Claude, local script, GitHub Action, or a human.

3.2 Artifact Registry

Every meaningful output should become a durable artifact. Artifacts are the shared memory of the system.

- task_doc
- review_notes
- implementation_summary
- diff_summary
- test_results
- ci_failure_analysis
- risk_assessment
- review_comment_plan
- merge_readiness_report

```
type Artifact = {
  id: string
  runId: string
  taskId: string
  producer: string
  type: "task_doc" | "review" | "test_results" | "ci_analysis" | "handoff" | "risk_report"
  content: unknown
  createdAt: string
  confidence?: number
  claims?: Claim[]
  dependsOn?: string[]
}
```

3.3 Event Log

The event log is the durable timeline. It lets agents and humans answer what happened and lets recipes react to important changes.

```
task.created
recipe.started
doc.reviewed
worktree.created
cursor_run.started
cursor_run.completed
pr.opened
ci.failed
ci.fixed
review.requested
review.completed
human.escalated
merged
```

Example reaction: ci.failed triggers the fix_ci recipe, which launches a Cursor repair agent, publishes ci_analysis, patches the branch, and resumes review when checks pass.

3.4 Claim and Decision Ledger

Agents should be able to publish evidence-backed claims and explicit decisions. This allows downstream agents to build on prior intelligence, challenge weak conclusions, and avoid redoing work.

```
type Claim = {
  id: string
```

```

    taskId: string
    producer: string
    claim: string
    confidence: number
    evidence: EvidenceRef[]
    status: "open" | "accepted" | "challenged" | "retracted"
  }

  type Decision = {
    id: string
    taskId: string
    decision: string
    reason: string
    approvedBy?: "human" | "policy" | "agent"
    createdAt: string
  }

```

Example claim: “The failing CI check is caused by missing TEST_DATABASE_URL” with confidence 0.82 and references to exact CI log lines and test files. A reviewer agent can challenge that claim with counter-evidence.

3.5 Handoff Protocol

The handoff is the most important primitive. It transfers ownership or readiness without free-form ambiguity.

```

type Handoff = {
  from: string
  to?: string
  taskId: string
  status: "done" | "blocked" | "needs_review" | "needs_human"
  summary: string
  changedFiles?: string[]
  artifacts: string[]
  openQuestions: string[]
  recommendedNextTask?: string
  risks: string[]
}

{
  "from": "cursor_implementer",
  "to": "review_cycle",
  "status": "needs_review",
  "summary": "Implemented artifact read API and added tests.",
  "changedFiles": ["internal/artifacts/store.go", "internal/mcp/artifacts_tool.go"],
  "artifacts": ["implementation_summary", "test_results"],
  "openQuestions": ["Should artifact size limits be configurable per repo?"],
  "risks": ["MCP response schema needs compatibility check."]
}

```

4. Ship’s Role in the Stack

Ship should become the workflow brain. It should not replace Tower, Cursor, GitHub, or CI. It should coordinate them through typed workflow primitives.

```

MCP clients
  Cursor / Claude / ChatGPT / CLI

Ship
  task units
  recipes
  event log
  artifact registry
  handoff protocol
  capability registry
  policies

Execution backends
  Cursor SDK
  Claude Code
  local scripts

```

```

    GitHub Actions
  Dev substrates
    Tower
    GitHub
    CI
    docs
```

Responsibility boundaries

Component	Owns
Tower	Registered repos, worktrees, branch/worktree mapping, local path discovery, PR/CI/review state snapshots.
Cursor SDK	Repo-focused implementation work: edit code, run commands, produce branch/PR/artifacts, stream agent status.
Ship	Tasks, recipes, run state, artifacts, events, policies, review cycles, CI repair loops, communication layer.
GitHub/CI	Issues, PRs, checks, comments, review state, merge status, release metadata.

5. MCP Primitives for the Communication Layer

The initial MCP API should be small and durable. Avoid low-level chat primitives. Expose typed operations that let any agent inspect, publish, and route work.

observe

Universal “what is going on?” tool for a repo, run, task, or PR.

```
observe({
  scope: "repo" | "run" | "task" | "pr",
  id: string
}) -> {
  currentState: string
  owner?: string
  activeAgents: string[]
  latestEvents: Event[]
  availableArtifacts: ArtifactSummary[]
  blockers: Blocker[]
  nextRecommendedActions: Action[]
}
```

publish

Typed record publication. This replaces noisy agent-to-agent chat for most coordination.

```
publish({
  taskId: string,
  type: "artifact" | "claim" | "decision" | "handoff" | "blocker",
  content: unknown
})
```

route

Ask Ship who or what should handle a need. Ship resolves based on capabilities, policies, repo profile, current state, and escalation rules.

```
route({
  taskId: string,
  need: "implement" | "fix_ci" | "review_security" | "review_tests" | "summarize" | "human_decision",
```

```

    context?: unknown
  }) -> {
    assigneeType: "agent" | "recipe" | "human" | "tool"
    assignee: string
    rationale: string
    nextToolCall?: unknown
  }

```

read_artifact / list_artifacts

Expose the artifact registry as first-class MCP tools and resources so agents can ground themselves in prior outputs.

```

list_artifacts({ taskId, type?, producer? })
read_artifact({ artifactId })

```

6. Repo-Focused Task Units and Recipes

The communication layer should support Ship's core model: task units are the primitives; recipes are ordered chains of task units.

```

Task units:
  create_task_doc
  review_task_doc
  ship_task_doc
  run_review_cycle
  next_action
  fix_ci
  address_review_comments
  merge_when_green

Recipe:
  ship_feature =
    create_task_doc
    -> review_task_doc
    -> ship_task_doc
    -> run_review_cycle
    -> next_action

```

Each task unit should publish artifacts, events, and handoffs. A recipe is just a reliable way to chain them with state, input/output passing, and policy gates.

Example: ship_feature communication trail

```

task.created
artifact.published(task_doc)
review.requested(architecture_reviewer)
artifact.published(review_notes)
decision.recorded(task_doc_approved)
worktree.created(tower/auth-cleanup)
cursor_run.started
cursor_run.completed
artifact.published(implementation_summary)
pr.opened
review_cycle.started
claim.published("tests cover token expiry regression")
ci.failed
handoff.published(status="blocked", recommendedNextTask="fix_ci")
ci.fixed
review.completed
handoff.published(status="done", recommendedNextTask="merge_when_green")

```

7. Policies and Guardrails

The system needs hard limits so agents do not loop endlessly or make risky autonomous decisions.

- **Max review cycles:** default to 3 automated cycles before human escalation.

- **Max CI repair attempts:** default to 3 attempts before human escalation.
- **No silent merge:** merging should require explicit policy or human approval.
- **No credential leakage:** never publish secrets into artifacts, claims, handoffs, or logs.
- **Evidence required for claims:** important technical claims should reference files, logs, PR comments, or test output.
- **Task ownership:** one active owner per task phase to reduce conflicting edits.
- **Human stop condition:** uncertainty, repeated failures, conflicting reviews, or risky migrations should trigger escalation.

8. Implementation Path

- 1 **V1:** task units, recipe runner, run state, artifacts, events, handoffs. No general messaging.
- 2 **V2:** capability registry, claim ledger, observe/publish/route MCP tools.
- 3 **V3:** subscriptions, policy-based routing, cross-repo coordination, team dashboard, richer analytics.

Recommended V1 schema

```
type ShipRun = {
  id: string
  recipe?: string
  status: string
  repo: string
  taskId: string
  currentStep?: string
  events: Event[]
  artifacts: ArtifactSummary[]
  handoffs: Handoff[]
  reviewCycles: number
  ciRepairAttempts: number
  prUrl?: string
  worktree?: { repo: string; branch: string; path: string }
}
```

9. Design Principles

- **Structured records over chat.** Chat is for humans; coordination should happen through artifacts, events, claims, decisions, and handoffs.
- **Repo work first.** Ship should focus on turning task docs into reviewed PRs, not generic multi-agent roleplay.
- **Agents are replaceable.** Cursor SDK is the first-class repo execution backend, but tasks and recipes should not be locked to one model.
- **Human escalation is a feature.** The system earns trust by knowing when to stop.
- **Make the process executable.** The goal is not more AI messages. The goal is a reliable workflow from idea to merged PR.

10. One-Sentence North Star

Ship should become the executable layer for how software work moves from idea to merged PR, while its communication layer lets agents and humans coordinate through durable shared intelligence

instead of noisy free-form chatter.

Appendix: Agent Prompt to Continue This Design

Use the following prompt to hand this idea to an implementation/design agent.

You are designing Ship, an MCP toolkit for repo-focused developer workflows. Ship has task units, recipes, run state,

Core primitives:

- Task: one named unit of workflow
- Recipe: ordered chain of task units
- Artifact: durable task output
- Event: timeline record
- Claim: evidence-backed assertion
- Decision: explicit design/policy choice
- Handoff: structured transfer of ownership/readiness
- Capability: what an agent/tool can do

Please produce:

1. Minimal V1 data model
2. MCP tools/resources
3. Storage layout
4. Event naming conventions
5. Handoff schema
6. Guardrails for max review/repair cycles
7. How this integrates with Tower and Cursor SDK
8. A staged implementation plan